Emp?rio da M?sica ? AI Customer Service Agent

Technical project documentation

Prototype conversational agent in Python for customer service at the fictional Emp?rio da M?sica store (Campo Grande, MS, Brazil). The project combines **Function Calling** for structured operational data with a simple lexical **RAG** layer to retrieve internal policies.

Features

? Product search by name, description, category, and maximum price.

? Price and inventory lookup from CSV files.

? Active promotion lookup (expired promotions are not offered).

? Order lookup with full customer-name validation.

? Store address, contact information, opening hours, and payment methods.

? Retrieval of policy excerpts covering exchanges, returns, shipping, warranty, and customer service.

? Message history during the terminal session.

? Error handling and escalation to human support when information is unavailable.

Architecture and Technical Decisions

```
Customer (CLI)
    |
    v
OpenAI Chat Completions + Function Calling
    |
    +---- search_products ------> Product CSVs + categories + promotions
    |
    +---- get_order_status -----> Orders CSVs + customers + items
    |                              (customer validation)
    |
    +---- store_information ----> Official store information
    |
    +---- search_store_policies -> Lexical retrieval from TXT policy manual (RAG)
```

Why a hybrid architecture?

CSV files contain structured and changeable data. Querying them through deterministic functions prevents the LLM from inventing prices, inventory levels, or order statuses. Policies are textual content, so they are retrieved through lexical similarity and provided to the model as context.

Framework, Model, and Provider

? **Python 3.10+** as the main language.

? **OpenAI Python SDK** and Chat Completions with Function Calling.

? Default model `gpt-4o-mini`, configurable through `OPENAI_MODEL`.

? The API key is read from `OPENAI_API_KEY` and is not stored in the code.

? A CLI was chosen to keep the scope focused and make the agent flow easy to test.

Retrieval and Prompt

The manual search splits the text into sections and ranks them by overlap of relevant terms. This is a transparent solution without additional vector infrastructure, suitable for the small manual provided. In production, it could evolve to embeddings, hybrid search (BM25 + vector), reranking, and page-level citations.

The prompt defines the persona, scope, mandatory lookup rules, and operating limits. The application does not allow the model to query order data without both an order number and the account holder's name.

Data Handling

? CSV files are read with Python's standard `csv` library.

? Prices are converted to and formatted as BRL.

? Only active products with positive inventory are returned.

? A promotion is considered only when `is_active` is true.

? An order lookup requires an exact match for the account holder's full name.

? The agent does not offer accessories because the manual states that the store sells only musical instruments.

How to Run

1. Prerequisites

? Python 3.10 or higher.

? An OpenAI API key with access to the configured model.

2. Installation

```
python -m venv .venv

# Windows
.venv\\Scripts\\activate

# macOS/Linux
source .venv/bin/activate

pip install -r requirements.txt
```

3. Configuration

Copy `.env.example` to `.env` and enter the key. The `.env` file must not be uploaded to GitHub.

```
OPENAI_API_KEY=your_key
OPENAI_MODEL=gpt-4o-mini
```

The application automatically loads the `.env` file using `python-dotenv`.

4. Start the Application

```
python app.py
```

CLI commands: `/limpar` resets the session history and `/sair` exits the application.

Interaction Examples

Complete examples are available in `examples/`. They cover catalog search, return policy, order validation, opening hours, and out-of-scope requests.

Known Limitations and Next Steps

1. The RAG layer is lexical and works well for a small manual, but it does not understand synonyms as well as embeddings.

2. The data source consists of static CSV files; production would require a transactional database and real-time updates.

3. Order authentication uses the full name and order number as a prototype. Production would require OTP or stronger authentication, attempt limiting, and audit logs.

4. There is no real integration with a payment gateway, carrier, WhatsApp, or CRM.

5. Conversation history is not persisted between executions; this could be added with SQLite, a retention policy, and user consent.

6. There is no automated evaluation of quality, latency, or cost; a production version should add unit tests, a reference-question set, and tracing.

Use of Coding Assistants

This prototype was structured with support from ChatGPT to interpret the requirements, plan the architecture, write and review parts of the implementation, and organize the documentation. The decisions, business rules, and limitations are documented to enable technical review.

Structure

```
emporio-musica-ai-agent/
??? app.py
??? requirements.txt
??? .env.example
??? README.md
??? data/
?   ??? products.csv
?   ??? promotions.csv
?   ??? customers.csv
?   ??? categories.csv
?   ??? order_items.csv
?   ??? orders.csv
?   ??? politicas_da_loja.txt
??? examples/
    ??? 01_catalogo.md
    ??? 02_devolucao.md
    ??? 03_pedido.md
    ??? 04_horarios.md
    ??? 05_fora_escopo.md
```

Transparency Note

The data supplied in the challenge is fictional. The conversation examples illustrate expected behavior; when the agent runs, natural-language responses may vary depending on the model. Prices, inventory, orders, and policies should always be checked against the supplied source files.